Cairo University
Faculty of Engineering
Department of Computer Engineering

Accelera

A Graduation Project Report Submitted

to

Faculty of Engineering, Cairo University

in Partial Fulfillment of the Requirements for the Degree of

Bachelor of Science in Computer Engineering

Presented by

Mohamed Ashraf	ID: 9220658	Nesma Osama	ID: 9220912
Mazen Adel	ID: 9220625	Mohamed Khater	ID: 9220713

Supervised by

Dr. Salma Abdel monem

April 17, 2026

Abstract

Accelera is a hybrid Python/C++ machine learning framework that combines a high-level Python interface with native execution components. The system is organized around five modules: Accelera Pipe, the Code Parallelizer, AutoPreprocessing, AutoML, Deployment, and the Benchmark Platform. Accelera Pipe represents machine-learning workflows as directed graph nodes for preprocessing, modelling, prediction, metric evaluation, branching, and merging. The Code Parallelizer analyzes C/C++ and supported Python code, extracts loop features, predicts parallelization opportunities, and emits OpenMP-annotated C++ code. This report documents the design, methodology, and experimental results available for the implemented modules, while retaining the required report structure for the remaining modules.

Arabic Abstract

Acknowledgment

Table of Contents

List of Figures

List of Tables

List of Abbreviations

AI	Artificial Intelligence
AST	Abstract Syntax Tree
CPU	Central Processing Unit
ML	Machine Learning
OpenMP	Open Multi-Processing
RSS	Resident Set Size
SVC	Support Vector Classifier
VMS	Virtual Memory Size

List of Symbols

X	Input feature matrix
y	Target vector
P	Source program
L	Extracted loop
$F(L)$	Feature representation of loop L

Contacts

Team Members

Name	University ID
Mohamed Ashraf	9220658
Nesma Osama	9220912
Mazen Adel	9220625
Mohamed Khater	9220713

Supervisor

Dr. Salma Abdel monem

Chapter 1: Introduction

1.1 Motivation and Justification

Accelera addresses the practical gap between the productivity of Python and the performance requirements of machine-learning workflows and loop-heavy data processing. A sequential script evaluates alternatives one after another and leaves memory lifetime and parallel execution decisions to the user. The project provides a graph-based pipeline backend and a code parallelization path to make these execution decisions explicit and reusable.

1.2 Project Objectives and Problem Definition

The project objective is to provide an integrated framework for constructing, executing, evaluating, and accelerating data-science workflows. The problem is to preserve a convenient Python-level programming model while enabling native graph scheduling, controlled intermediate-data lifetime, and OpenMP-backed execution for supported custom code.

1.3 Project Outcomes

The project is organized into the following modules:

- Accelera Pipe for graph-based machine-learning workflows.
- Code Parallelizer for source analysis and OpenMP generation.
- AutoPreprocessing for tabular, text, and image data preparation.
- AutoML for model selection and optimization.
- Deployment and Benchmark Platform components for serving and evaluation.

1.4 Document Organization

Chapter 2 reserves the required market visibility study. Chapter 3 contains the literature-survey structure. Chapter 4 documents the system design and implemented module methodology. Chapter 5 presents testing and verification results. Chapter 6 concludes the report and records future work. The appendices retain the required development, user-guide, and feasibility-study sections.

Chapter 2: Market Visibility Study

2.1 Targeted Customers

2.2 Market Survey

2.3 Business Case and Financial Analysis

Chapter 3: Literature Survey

3.1 Background and Previous Work

3.2 Comparative Study of Previous Work

3.3 Implemented Approach

Chapter 4: System Design and Architecture

4.1 Overview and Assumptions

The system combines Python-facing APIs with native execution components. The design assumes that machine-learning workflows can be represented as a graph of operations and that only a restricted, analyzable subset of custom source code should be translated to parallel C++.

4.2 System Architecture

4.2.1 Block Diagram

The implemented system is organized around graph execution, source-code parallelization, automated preprocessing, model-selection, deployment, and benchmarking components. The following module sections describe the material available in the previous thesis.

4.3 Accelera Pipe Module

4.3.1 Module Responsibility

Accelera Pipe is implemented under `accelera/src/accelera_pipe/`. Its responsibility is to provide a Python builder API over a C++ graph backend. The user writes a pipeline in Python, but the graph structure, node scheduling, branch execution, and intermediate memory lifetime are handled by the native backend.

The core files are:

- `core/pipeline_base.py`: owns or receives a native `graph.Graph` instance and provides shared save/load behavior.
- `core/pipeline.py`: implements the public builder API: `preprocess`, `model`, `predict`, `metric`, `merge`, and `branch`.
- `core/executed_graph.py`: wraps a trained graph for repeated inference.
- `core/metric.py`: defines the metric abstraction.
- `wrappers/`: contains supervised and unsupervised metric adapters.

4.3.2 Graph Execution Model

Each pipeline operation becomes a graph node. A preprocessing node transforms the input matrix, a model node fits or applies an estimator, a prediction node performs inference, a metric node evaluates the prediction, and a merge node combines multiple branch outputs. The graph is built incrementally, so the Python API remains fluent while the backend receives enough structure to execute independent branches in parallel.

Table 4.1: Accelera Pipe builder methods

Method	Node type	Main role
`preprocess(name, func)`	PREPROCESS	Transform input data
`model(name, model)`	MODEL	Fit or apply estimator
`predict(name, test_data)`	PREDICT	Run model prediction
`metric(name, metric)`	METRIC	Score predictions
`merge(name, method)`	MERGE	Combine branch outputs
`branch(name, ...)`	Multiple	Create alternative graph paths

The pipeline returns two objects after training: the metric or prediction results and an `ExecutedGraph`. The executed graph contains the fitted objects required for inference. This separation is important because training uses temporary arrays, validation targets, and candidate branches, while inference should keep only the selected executable state.

Algorithm 4.1: Pipeline training and executed graph construction

```
Input: training data X, target y, graph G, selection strategy S
Output: results R, executed graph E

1:  Attach X and y to the input node of G
2:  Execute ready preprocessing, model, prediction, metric, and merge nodes
3:  For each candidate path:
4:      collect validation metric and path metadata
5:  Select path P according to strategy S
6:  Clone the selected fitted nodes from P into a compact executed graph E
7:  Remove temporary training arrays and non-required execution data from G
8:  Return results R and executed graph E
```

4.3.3 Branching and Node Sharing

Branching is one of the main reasons Accelera Pipe uses a graph rather than a linear pipeline object. A user may compare several preprocessors, several models, or complete candidate sub-pipelines. The backend can then schedule independent branches concurrently. A key optimization added to the branch construction step is duplicate

first-node sharing. If a branch starts with the same single preprocessing object as another branch, the graph creates the node once and connects the later branch consumers to the same output. This avoids running identical first-stage transformations multiple times.

The sharing rule is intentionally conservative. It is applied only to single branch nodes, the first node of a branch list, or a list containing one item. Later duplicated nodes inside two different lists are not merged automatically, because by that point their inputs may already be different. For example, `[p1, p2, p3]` and `[p4, p2, p3]` do not share `p2`; the first path feeds `p2` with `p1(X)`, while the second feeds it with `p4(X)`.

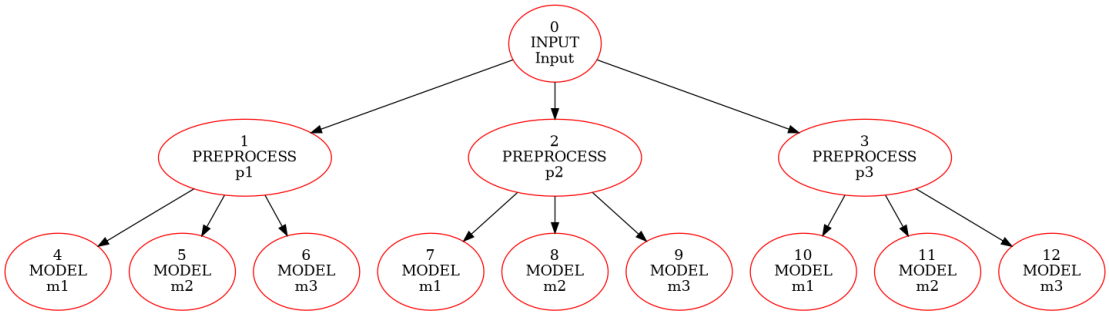

Figure 4.1: Branch graph before duplicate first-node sharing

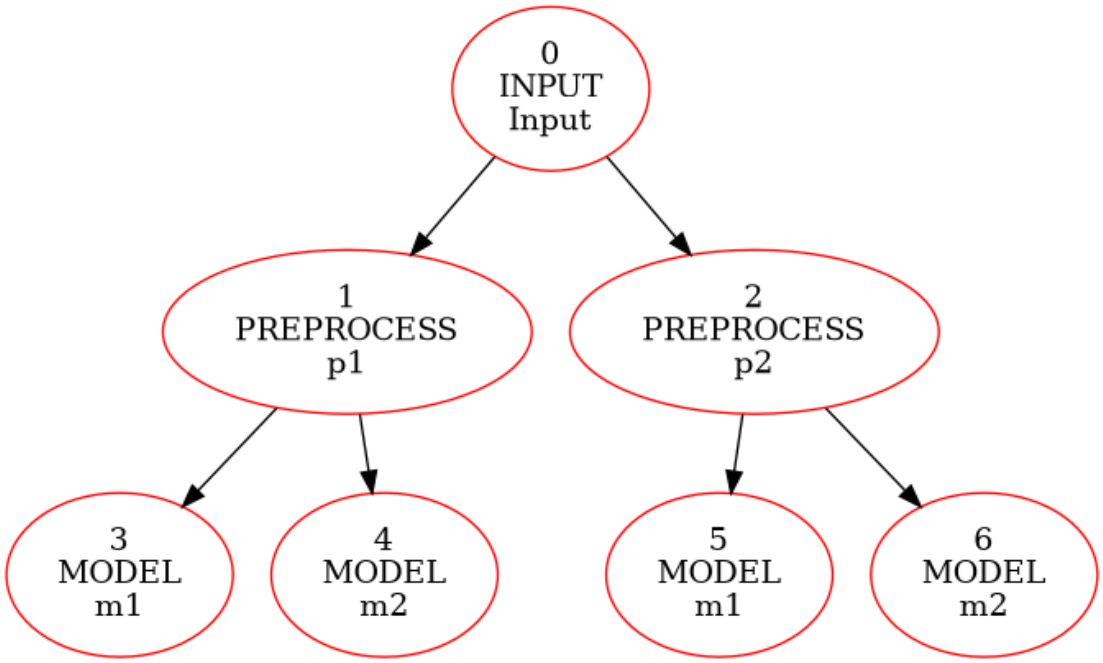

Figure 4.2: Branch graph after duplicate first-node sharing

4.3.4 Memory-Lifetime Optimizations

The largest memory pressure in Accelera Pipe appears during branch execution. Each preprocessing node can produce a transformed training array, and that array may be consumed by several downstream model nodes. The backend therefore tracks how

many consumers still need each intermediate output. When the last consumer finishes, the output becomes dead and can be released.

Algorithm 4.2: Consumer-count based intermediate release

```
Input: executed node N, graph consumer count table C
Output: graph with dead temporary arrays cleared

1:  for each source node S that feeds N do
2:      C[S] = C[S] - 1
3:      if C[S] == 0 and S is releasable then
4:          clear temporary data stored by S
5:      end if
6:  end for
7:  if N is a model node and fit has completed then
8:      clear the training input stored only for fitting N
9:  end if
```

Several smaller optimizations support this lifetime policy. Cache execution is optional and disabled by default using `cache=False`, because retaining node data can increase memory growth on large datasets. Model nodes release their training arrays after fitting because the fitted estimator is sufficient for inference. The backend also separates output-container allocation from input copying. For preprocessing, the helper `shouldCopyPreprocessInput` decides whether a defensive copy is needed. Scikit-learn transformers normally return transformed data without mutating the original input, so unnecessary copies can be skipped for those objects. Custom functions remain protected when mutation is possible.

4.3.5 Saving and Loading Pipelines

Pipeline objects can be saved as pickle files. Native graph pickling is provided through the PyBind binding for `graph.Graph`, where C++ exposes a `py::pickle` state pair. During `pickle.dump`, Python asks the bound object for its registered pickle state, and the C++ graph returns a Python dictionary that describes the graph structure and stored objects. During load, the inverse state function rebuilds the graph object.

Custom preprocessing functions are handled by storing source code when normal pickle cannot serialize the function object directly. The save path extracts a top-level function or simple lambda source, and the load path executes that source inside a controlled namespace to recover the callable. Functions with closures are rejected because their external captured variables would not be available from source text alone.

4.4 Code Parallelizer Module

4.4.1 Module Responsibility

The Code Parallelizer transforms supported loop-heavy source code into OpenMP-annotated C++. The main Python orchestration file is `accelera/src/utils/parallelizer.py`. Python input is first lowered by `accelera/src/utils/py2cpp_converter.py`. Native loop extraction and pragma insertion are implemented in C++ under `src/ast/` and `src/utils/code_parallelizer_utils.cpp`. The module can process files or in-memory code strings.

The workflow has four stages:

1. Convert supported Python code to C++ when the input is Python.
2. Use Clang-based analysis to extract loops and loop metadata.
3. Predict the loop class using a classifier and rule checks.
4. Insert OpenMP pragmas and return the transformed C++ code.

4.4.2 Loop Analysis and Pragma Selection

Loop extraction records syntactic and semantic features such as loop nesting, array access, assignments, reduction patterns, loop-carried dependencies, function calls, and scalar updates. These features are passed to the classifier service. The classifier predicts one of three classes: `none`, `parallel_for`, or `reduction`. The rule layer then validates the predicted class before emitting the pragma.

Algorithm 4.3: Loop classification and OpenMP annotation

```
Input: source program P
Output: OpenMP-annotated C++ program P'

1:  if P is Python then
2:      convert the supported Python subset to C++
3:  end if
4:  Parse C++ source using the Clang frontend action
5:  for each extracted loop L do
6:      compute structural and dependency features F(L)
7:      request class c from the OpenMP classifier
8:      if c is parallel_for and dependency checks pass then
9:          insert "#pragma omp parallel for"
10:     else if c is reduction and reduction variable is valid then
11:         insert "#pragma omp parallel for reduction(...)"
12:     else
13:         keep the loop sequential
14:     end if
15: end for
16: Return the rewritten C++ source
```

4.4.3 Classifier Model and Rule-Based Decision

The classifier assets are stored under `models/open_mp_classifier/`. The service loads `model.pkl` and `label_encoder.pkl`, exposes a FastAPI prediction endpoint, and maps loop features to one of the supported pragma classes. The trained model is an `XGBClassifier` over a fixed 40-feature loop representation.

An earlier generator trial attempted to generate pragma decisions using a neural sequence model. The model used a custom PyTorch encoder-decoder architecture: a bidirectional LSTM encoder, an attention-based LSTM decoder, teacher forcing during training, cross-entropy loss, Adam optimization, gradient clipping, and a custom BPE tokenizer. This approach was rejected for two scientific reasons. First, the available dataset contained about 15,000 samples, which was not enough for a sequence model to generalize reliably to unseen program structures; the model overfit easily. Second, generating extra training programs with AI tools risked injecting generator-specific bias, which would lower generalization on real user code. The final approach therefore uses explicit loop features, a classifier, and rule-based safety checks.

4.4.4 Python-to-C++ Converter Support

The Python converter intentionally supports a restricted subset. This makes the generated C++ predictable enough for the parallelizer and avoids silently miscompiling dynamic Python behavior.

Table 4.2: Supported operations in the Python-to-C++ converter

Category	Supported forms	Notes
Values and names	Numeric constants, names, simple lists	Dynamic objects are outside the subset.
Arithmetic	`+`, `-`, `*`, `/`, `%`, `**`	Power maps to `std::pow`.
Boolean logic	`and`, `or`, `not`	Emitted as C++ boolean expressions.
Comparisons	`<`, `<=`, `>`, `>=`, `==`, `!=`	Chained comparisons are restricted.
Calls	Supported built-ins and direct calls	Unsupported calls raise converter errors.
Indexing	Array-style subscripting	Slice semantics are not supported.
Assignment	Simple assignment and selected augmented assignment	Used for loop updates and reductions.
Control flow	`if/else`, `return`, `for range(...)`	Main loop form used by the parallelizer.

Category	Supported forms	Notes
Functions	Plain function definitions	Decorators, closures, and dynamic dispatch are outside the safe subset.

4.4.5 Integration with Accelera Pipe

The parallelizer is integrated into Accelera Pipe for custom preprocessing functions and custom transformer instances. If a preprocessing function is a user-defined Python function, the pipeline attempts to optimize it through the parallelizer. If the object is a custom transformer, the transform method can be optimized and attached back to the instance. This allows the user to write a normal Python preprocessing step while the system compiles the loop-heavy part into a Python-callable C++ extension.

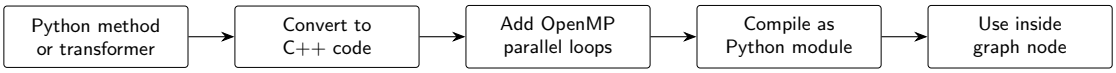

Figure 4.3: Integration flow for compiled preprocessing inside Accelera Pipe

The integration is applied during graph construction rather than as a separate user-facing execution stage. A custom preprocessing method is inspected, lowered to the supported C++ subset, analyzed for loop-level parallelism, compiled into a Python-callable native module, and then stored back inside the preprocessing node. As a result, the graph still executes a normal preprocessing node, but the callable held by that node may be a compiled OpenMP implementation instead of the original Python loop.

4.5 AutoPreprocessing Module

4.5.1 Module Overview

In the data science lifecycle, data understanding, exploratory data analysis (EDA), and data preparation represent major stages that involve a large amount of repetitive work. In this module, the aim is to reduce this repetitive effort by introducing a rule-based system that automates these processes.

The module is designed to work based on the type of data, whether tabular, text, or image. For each data type, the corresponding data understanding and preprocessing steps are applied to ensure that the data is properly prepared and suitable for input into machine learning and deep learning models.

The system also generates a final report that provides the user with information about the dataset and the applied preprocessing steps. For example, in tabular data, this includes details such as missing values, number of duplicates, dataset size, and

column types. In addition, it describes the preprocessing steps that were applied to the data and presents the final results after processing.

4.5.2 Supported Dataset Types

- Tabular Dataset
- Text Dataset
- Image Dataset

4.5.3 Supported Tasks

- Classification and Regression for tabular dataset
- Classification for text dataset
- Classification image dataset

4.5.4 Tabular Dataset Training

The meaning of The tabular dataset in this module is structured data organized in rows and columns, where features can be either categorical or numerical.

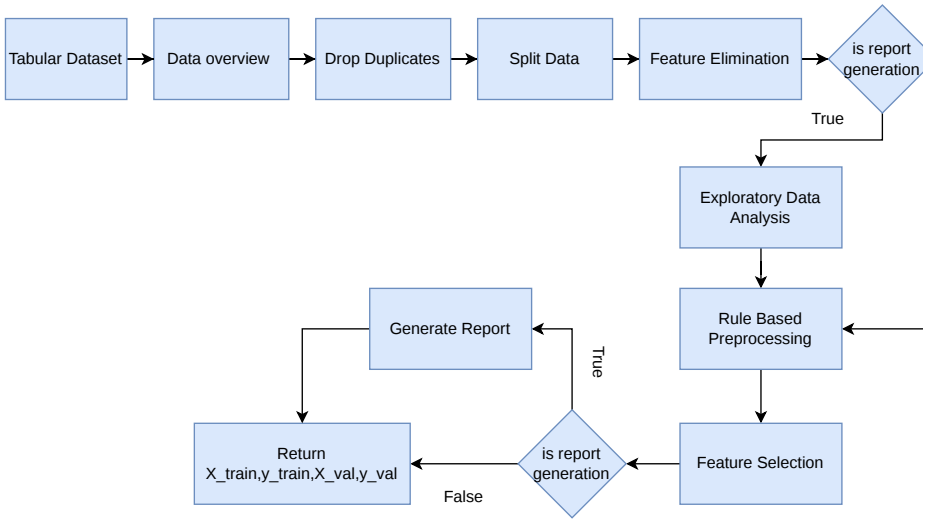

Figure 4.4: Tabular data Preprocessing Digram

Training Preprocessing Steps

1. Data Input and User Configuration Initialization.
2. Get data overview.
3. Drop duplicates.
4. Split dataset.
5. Get training data information.
6. Specify columns to drop.
7. Drop selected columns.
8. Detect the type of each column based on the training data.
9. Generate visualizations of the training data based on column types and task type.
10. Apply feature preprocessing techniques for each column based on its detected type.
11. Apply target preprocessing based on the selected task.
12. Feature Selection.
13. Save all files required for preprocessing the test data.
14. Generate Report if user enable report generation.
15. Return `X_train`, `y_train`, `X_val`, `y_val`.

Data Input and User Configuration Initialization: The preprocessing pipeline is initialized through the class constructor. During initialization, all user-defined parameters are assigned and validated to ensure consistency and correctness before executing any preprocessing steps. These validations help prevent invalid configurations such as incorrect problem types, wrong target column name, or invalid parameter values, which could otherwise lead to runtime errors or incorrect model behavior.

Table 4.3: Classical Training Data Preprocessing Parameters - Description

Parameter	Description
df	The dataframe of the dataset.
target_col	The name of the target column in the dataset.
problem_type	Classification or Regression task type.
folder_path	Path to store reports, charts, and serialized objects.
val_size	Percentage of data used for validation split.
random_state	To ensure reproducible data splitting.
cardinality_threshold	Threshold for separating low and high cardinality categorical features.
max_unique_ordinal	Maximum number of unique values to treat a column as ordinal.
missing_threshold	Threshold for dropping columns based on missing values ratio.
columns_need_to_drop	List of user-specified columns to remove.
feature_importance_threshold	Threshold to select the most relevant features.
is_report	Enables or disables preprocessing report generation.

Table 4.4: Classical Training Data Preprocessing Parameters - Constraints

Parameter	Allowed Values / Constraints
df	pandas DataFrame (must include required columns)
target_col	String; must exist in dataframe columns
problem_type	"classification" or "regression"
folder_path	String path
val_size	Float: $0 < val\_size \leq 0.5$
random_state	Integer or None
cardinality_threshold	Integer ≥ 0
max_unique_ordinal	Integer ≥ 0
missing_threshold	Float or Integer: $0 \leq x \leq 1$
columns_need_to_drop	List of strings
feature_importance_threshold	Float or Integer: $0 \leq x \leq 1$
is_report	Boolean (True / False)

Data Overview: This step is responsible for generating a comprehensive statistical and structural summary of the dataset before preprocessing begins, helping to create a detailed report.

1. **Capture initial data snapshot:** The first five rows of the original dataset are stored to preserve the original structure before any modifications.
2. **Standardize text data:** All string-based values in the dataset are converted to lowercase, ensuring consistency in categorical values.
3. **Capture transformed snapshot:** After applying lowercase transformation, another preview of the dataset is stored for comparison purposes.
4. **Generate dataset structure information:** to capture column types, non-null counts, and memory usage.
5. **Separate numerical and categorical features:** Columns are divided based on data type into numerical and categorical subsets using `select_dtypes()`.

6. **Compute descriptive statistics:**
    - Numerical features: mean, standard deviation, min, max, quartiles.
    - Categorical features: frequency, unique values, top category.
7. **Compute missing values:** The number of missing values per column is calculated.
8. **Detect duplicate rows:**
    - Total number of duplicate rows is computed.
    - Percentage of duplicated rows is calculated relative to dataset size.
9. **Store dataset shape:** The dataset dimensions (rows and columns) are recorded.
10. **Save all results into report structure:** All computed statistics, snapshots, and metadata are stored for later reporting and visualization.

Algorithm 4.4: Data Overview Generation

```
Input: Dataframe D
Output: Stored data overview in report object

Extract the first rows of D as an initial preview
Convert string values in object-type columns to lowercase
Extract dataframe structure information using the info function
Split D into numerical features D_num and categorical features D_cat
if D_num is not empty then
    compute numerical statistics: mean, standard deviation, min, max, quartiles
end if
if D_cat is not empty then
    compute categorical statistics: unique values, frequencies, top category
end if
Compute missing values for each column
Compute duplicate row count and duplicate percentage
Store dataframe shape: number of rows and columns
Save head, info, statistics, missing values, duplicates, and shape in report_data
Return the updated report object
```

Drop duplicates: Duplicate rows are removed from the dataset to prevent the model from overfitting to repeated examples and to ensure a more reliable evaluation process.

After performing this step, the system reports key statistics to the user, including the total number of duplicate rows detected, the percentage of duplicated data, and the resulting dataset shape after removal. This allows the user to verify that the dataset no longer contains redundant entries.

Split data: Split the data into `X_train`, `X_val`, `y_train`, and `y_val` using the `train_test_split` function from Scikit-learn. The split follows the configured `test_size` and `random_state` values.

Get training data information: For each column, calculate the required information, such as the percentage of missing values,number of unique values, the mean and median for numerical columns, and the mode value

Specify Columns to Drop: A column is dropped if:

- It is included in the user-provided `columns_need_to_drop` parameter.
- The percentage of missing values is greater than the user-provided `missing_threshold`.
- The column is a constant value.

Drop Selected Columns: The specified columns are removed from both `X_train` and `X_val`.The dropped column names are serialized and saved as a pickle file to ensure consistency when applying the same preprocessing steps on future data.

Detect the type of each column based on the training data: Each feature is automatically assigned a column type. The detected type determines the preprocessing techniques that will be applied later in the pipeline.

- **Binary**: Columns containing exactly two unique values or boolean values. Examples: “yes/no”, “1/0”, “True/False”.
- **Ordinal**: Integer columns with a small number of unique values less than or equal `max_ordinal_unique`, where values have a meaningful order. Example: $1, 2, 3, 4$ (ratings, levels).
- **Numerical**: Integer columns with a number of unique values greater than `max_ordinal_unique`, typically representing continuous or high-variance numeric data. Examples: age, number of transactions.
- **Continuous**: Floating-point columns representing real-valued measurements. Examples: height = 175.5, price = 19.99, temperature = 36.6.
- **Low-Cardinality Categorical**: Object (string) columns with a number of unique values less than or equal to `cardinality_threshold`. Examples: product categories.

Table 4.5: Detected column types.

Column type	Detection rule
Binary	Columns containing exactly two unique values or boolean values.
Ordinal	Integer columns with a number of unique values less than or equal to `max_unique_ordinal`.
Numerical	Integer columns with a number of unique values greater than `max_unique_ordinal`.
Continuous	Floating-point columns.
Low-Cardinality Categorical	Object columns with a number of unique values less than or equal to `cardinality_threshold`.
High-Cardinality Categorical	Object columns with a number of unique values greater than `cardinality_threshold`.
Other	Columns with unsupported data types that are excluded from the preprocessing pipeline.

- **High-Cardinality Categorical**: Object (string) columns with a number of unique values greater than `cardinality_threshold`. Examples: product names, cities.
- **Other**: Columns with unsupported or non-standard data types that are excluded from the preprocessing pipeline.

Algorithm 4.5: Column Type Detection and Preprocessing Assignment

```
Input: Training features X_train
Output: Column groups and assigned preprocessing rules

 for each column in X_train do
     if column is binary then
         assign type Binary
         assign most frequent imputation and ordinal encoding
     else if column is integer then
         if unique values <= maximum ordinal threshold then
             assign type Ordinal
             assign most frequent imputation and robust scaling
         else
             assign type Numerical
             assign median imputation and robust scaling
         end if
     else if column is floating-point then
         assign type Continuous
         assign median imputation and robust scaling
     else if column is categorical then
         if cardinality <= cardinality threshold then
             assign type Low Cardinality Categorical
```

```
            assign most frequent imputation and binary encoding
        else
            assign type High Cardinality Categorical
            assign most frequent imputation and frequency encoding
        end if
    else
        mark column as unsupported and schedule it for removal
    end if
    store column type and preprocessing steps in the report
end for
Return all categorized column groups
```

Generate visualizations of the training data based on column types and task type: When the `is_report` flag is enabled, the tabular preprocessing report produces a set of visualizations corresponding to each detected feature type and task type.

- **Binary / categorical features with classification targets:** The report produces these graphs
  - a null-vs-non-null pie chart,
  - a category count plot ,
  - a category vs target count plot .
- **Binary / categorical features with regression targets:** The report produces these graphs
  - a null-vs-non-null pie chart,
  - a category count plot ,
  - a box plot of the continuous target for each category.
- **Ordinal features with classification targets:** The report produces these graphs
  - a null-vs-non-null pie chart,
  - an ordered count plot,
  - an ordered count plot split by class label.
- **Ordinal features with regression targets:** The report produces these graphs
  - a null-vs-non-null pie chart,
  - an ordered count plot,
  - a box plot of the regression target for each ordinal level.

- **Numerical / continuous features with classification targets:** The report produces these graphs
  - a null-vs-non-null pie chart,
  - a histogram with a kernel density estimate,
  - a box plot of the feature grouped by class label.
- **Numerical / continuous features with regression targets:** The report produces these graphs
  - a null-vs-non-null pie chart,
  - a histogram with a kernel density estimate,
  - a scatter plot of feature value against the regression target.
- **Target variable for classification:** The report produces these graphs
  - a null-vs-non-null pie chart,
  - a target class frequency count plot.
- **Target variable for regression:** The report produces these graphs
  - a null-vs-non-null pie chart,
  - a histogram with a kernel density estimate,
  - a box plot of the target distribution.
- **Numeric correlation matrix:** The report generates a heatmap of pairwise correlations across all numeric features.

Interpretation of visualization types:

- **Null vs non-null pie chart:** shows the proportion of missing values in a feature, helping detect data quality issues.
- **Category count plot:** shows the frequency distribution of categorical values and reveals imbalance or rare categories.
- **Category vs target count plot:** shows how each category relates to the target variable,helps identify whether certain categories are strongly associated with specific target outcomes.
- **Ordered count plot:** shows the frequency of ordinal values while preserving their order, helping visualize how samples are distributed across ordered levels.
- **Ordered count plot split by class:** shows how different classes are distributed across ordered levels (e.g., 1 to 4), helping identify whether certain classes tend to appear more in specific ordinal levels.

- **Box plot (category/ordinal vs target):** shows the distribution of a continuous target across groups, highlighting differences, variance, and outliers.
- **Histogram + KDE:** shows the distribution shape of a feature or target, including skewness, modality, and spread.
- **Scatter plot (feature vs target):** shows relationships or trends between numerical features and regression targets, including linear/non-linear patterns.
- **Box plot grouped by class label:** shows how numerical features vary across classes and helps detect outliers.
- **Correlation heatmap:** shows pairwise linear relationships between numerical features, helping detect multicollinearity and redundancy.
- **Target frequency plot (classification):** shows class imbalance, which is critical for model selection and evaluation strategy.
- **Target histogram (regression):** shows distribution of target values, helping detect skewness and outliers.

Table 4.6: Feature preprocessing applied by column type.

Column type	Applied preprocessing
Numerical / continuous	Imputation with `SimpleImputer(strategy="median")`, followed by `RobustScaler()`.
Binary	Most-frequent imputation followed by ordinal encoding with unknown values mapped to -1.
Low-cardinality categorical	Most-frequent imputation followed by binary encoding.
Frequency-encoded categorical	Most-frequent imputation followed by a frequency encoding transform.
Ordinal	Most-frequent imputation followed by robust scaling.

Apply feature preprocessing techniques for each column based on its detected type:

Apply target preprocessing based on the selected task: Target preprocessing changes based on problem type (classification / regression):

- **Classification:** Fill missing target values with the training-mode label, then encode the labels with `LabelEncoder`.
- **Regression:** Fill missing target values with the training median, then robust scale the target using `RobustScaler`.

Table 4.7: Justification of preprocessing techniques by feature type.

Preprocessing choice	**Justification**
Median imputation	Robust to outliers compared to mean imputation, making it suitable for skewed numerical distributions.
Robust scaling	Reduces the influence of outliers by scaling using the interquartile range instead of mean and variance.
Most-frequent imputation	Preserves the natural distribution of categorical variables by replacing missing values with the dominant category.
Binary encoding	Efficient representation for categorical variables, reducing dimensionality from $O(k)$ in one-hot encoding to $O(\log k)$ while preserving information.
Frequency encoding	Captures category importance based on occurrence frequency, reducing sparsity and preserving distributional signals.
Ordinal handling	Preserves inherent ordering information in the feature (e.g., 1 < 2 < 3 < 4), unlike nominal encodings.
Unknown category mapping (-1)	Ensures robustness during inference by handling unseen categories without breaking the encoding schema.

Algorithm 4.6: Target Variable Preprocessing

```
Input: Target y_train, y_val and problem type
Output: Processed targets and saved target metadata

if problem type is Classification then
    replace missing target values using the most frequent target value
    fit a LabelEncoder on the training target
    transform training and validation targets
    save the fitted LabelEncoder
    record classification target preprocessing in the report
else if problem type is Regression then
    replace missing target values using the median target value
    fit a RobustScaler on the training target
    scale training and validation targets
    save the fitted RobustScaler
    record regression target preprocessing in the report
end if
Store processed target samples in the report
Save target metadata for inference
Return processed training and validation targets
```

Feature Selection: Feature selection is performed using Mutual Information to measure the dependency between each feature and the target variable. For classification tasks, Mutual Information Classification is used, while Mutual Information Regression is applied for regression problems. Each feature is assigned an importance score based on its Mutual Information value. The features are then ranked in descending order according to their scores. Only features whose scores exceed a predefined threshold feature_importance_threshold are selected for model training. If no feature satisfies the threshold condition, an exception is raised to ensure model validity. The selected features are then used to filter both the training and validation sets. Finally, the list of selected features is saved for reproducibility and later use during inference.

Save all files required for preprocessing the test data: The preprocessing pipeline persists the objects required for later inference and reuse:

Table 4.8: Saved preprocessing artifacts for test-time reuse.

Artifact	Purpose
`training_preprocessor.pkl`	The fitted feature preprocessor.
`feature_names.pkl`	The output feature names produced by the feature preprocessor.
`target_preprocessor.pkl`	The fitted target encoder or scaler.
`target_info.pkl`	Information about the target, including target name, problem type, mode, and median.
`bool_type_col.pkl`	Boolean columns that are converted to integer values during preprocessing.
`data_columns.pkl`	The column names of the input DataFrame before preprocessing.
`col_drop.pkl`	The names of the dropped columns.
`selected_features.pkl`	The names of the final selected columns.

Generate Report (Optional): When `is_report` is enabled, the `TabularPreprocessingReport` class generates an HTML report from the collected `report_data`. The report documents the complete preprocessing workflow, including data overview statistics, missing values, duplicate analysis, train-validation split details, removed columns,

generated graphs, preprocessing decisions for each feature, and previews of the transformed training and validation datasets. This allows users to inspect and understand all preprocessing steps applied before model training.

4.5.5 Tabular Dataset Testing

Testing Preprocessing Steps The testing preprocessing workflow is designed to reuse the exact training preprocessing artifacts and transformations.

1. Verify that the required artifact files exist: target preprocessor, training preprocessor, `data_columns.pkl`, `col_drop.pkl`, `target_info.pkl`, `selected_features.pkl`, `feature_names.pkl`and `bool_type_col.pkl`.
2. Load the saved dataset columns, dropped columns, target information, boolean column list,selected features, training preprocessor, and target preprocessor.
3. Confirm `target_info` contains a valid `col_name` and `problem_type` (classification or regression).
4. Confirm `target_preprocessor` and `training_preprocessor` is not none.
5. Ensure that the test dataset column order matches the saved training columns. If the target column is missing, testing proceeds in `features_only` mode.
6. Normalize string values in the test dataset by lowercasing them, matching the same text normalization used during training preprocessing.
7. Separate the test DataFrame into `X_test` and `y_test` when `features_only` mode is false.
8. Convert the columns listed in `bool_type_col.pkl` to integer data type.
9. Drop the same columns identified during training using `col_drop.pkl`.
10. Apply the saved `training_preprocessor.pkl` transformer to `X_test`.
11. Filter X_{train} to keep only the selected features.
12. Apply target preprocessing when when `features_only` mode is false:
    - For classification, fill missing labels with the training-mode value, then apply the saved label encoder.
    - For regression, fill missing values with the training median, then apply the saved standard scaler.
13. Return the preprocessed test features and labels when `features_only` is set to `False`; otherwise, return the preprocessed test features and `None` for the labels.

This ensures the test pipeline uses the same feature transformation and target encoding logic as training.

Table 4.9: Classical Testing Data Preprocessing Parameters

Parameter	Description
df	The dataframe of the test dataset It can cotain target column also can not in this case enable feature_only mode.
folder_path	Path to load saved preprocessing artifacts.

4.5.6 Text Dataset Training

The text dataset consists of unstructured data containing a single text column used for natural language processing tasks.

Training Preprocessing Steps

1. Constructor Initialization and Input Validation.
2. Initialize NLP dependencies.
3. Generate data overview and split dataset.
4. Generate optional graphs.
5. Handle missing text values.
6. Apply text preprocessing and tokenization.
7. Transform text using TF-IDF vectorization.
8. Encode target labels.
9. Save preprocessing artifacts.
10. Generate preprocessing report (optional).
11. Return processed datasets.

Constructor Initialization and Input Validation: The constructor initializes all required parameters. During initialization, the method performs validation checks to ensure that the text column is correctly defined, is different from the target column, and has the appropriate data type. It also verifies that the target variable is suitable for classification tasks. Additionally, unnecessary columns are removed, required NLP resources are downloaded, and all configuration parameters are stored for later use.

Table 4.10: Text Training Data Preprocessing Parameters

Parameter	Description
`df`	Input pandas DataFrame containing the dataset.
`target_col`	Name of the target column used for classification.
`text_col`	Name of the column containing raw text data for NLP processing.
`folder_path`	Directory path used to store preprocessing outputs and reports.
`val_size`	Fraction of data used for validation split (default: 0.2).
`random_state`	Random seed for reproducible train-validation splitting (default: 42).
`tfidf_max_features`	Maximum vocabulary size for TF-IDF vectorization (default: 500).
`tfidf_ngram`	N-gram range for TF-IDF vectorizer (default: (1,2)).
`tfidf_max_df`	Maximum document frequency threshold for TF-IDF (default: 0.85).
`tfidf_min_df`	Minimum document frequency threshold for TF-IDF (default: 3).
`is_report`	Boolean flag to enable or disable report generation.

Initialize NLP dependencies: Required Natural Language Processing resources such as tokenizers and stopword lists are downloaded and initialized to enable text preprocessing operations.

Generate data overview and split dataset: A statistical overview of the dataset is generated, including missing values, duplicates, and descriptive statistics. After that, the dataset is split into training and validation sets using the configured split ratio.

Generate optional graphs: If enabled, the system generates visualizations such as text distribution plots and target distribution plots to better understand dataset characteristics.

Table 4.11: Optional Graph Generation in Text Preprocessing

Graph Type	Purpose
Null vs Non-Null Distribution	Shows missing value ratio in text and target columns.
Target Distribution Plot	Displays class frequency distribution for classification tasks.
Top-K Word Frequency Plot	Shows the most frequent 7 words in the corpus.

Handle missing text values: All missing values in the text column are replaced with empty strings.

Apply text preprocessing and tokenization: Raw text is cleaned and transformed using a custom tokenizer. This includes lowercasing, removing noise characters, handling contractions, removing stopwords, and applying stemming to normalize tokens.

Algorithm 4.7: Custom Text Tokenizer

```
Input: Raw text string
Output: List of cleaned and stemmed tokens

1:  Initialize English stopwords and remove {"not", "no"} from stopword list
2:  Initialize PorterStemmer
3:  Convert text to lowercase and strip whitespace
4:  Normalize special apostrophes to signle qoute
5:  Replace contractions:
        "can't" to "can not"
        "won't" to "will not"
        "shan't" to "shall not"
6:  Expand "n't" patterns into " not "
7:  Insert space between digits and letters
8:  Remove all characters except letters, digits, and apostrophes
9:  Collapse multiple spaces into a single space
10: Split text into tokens
11: Remove surrounding quotes from each token
12: Remove stopwords (except "not" and "no")
13: Remove tokens with length less than ar equal 1
14: Apply Porter stemming to remaining tokens
```

```
15: Return final token list
```

Transform text using TF-IDF vectorization: The cleaned text is converted into numerical feature vectors using a TF-IDF vectorizer, which captures the importance of words based on their frequency across documents.

Encode target labels: Categorical target labels are converted into numerical form using label encoding. Missing target values are filled using the most frequent class.

Save preprocessing artifacts: All fitted preprocessing objects such as the TF-IDF vectorizer and label encoder are saved to disk for reuse during inference.

Generate preprocessing report (optional): If enabled, a detailed report is generated containing preprocessing steps, dataset statistics, and visualizations for analysis and reproducibility.

Return processed datasets: The final processed training and validation datasets are returned, including transformed features and encoded labels, ready for model training.

Algorithm 4.8: Common Preprocessing Pipeline

```
Input: Raw dataframe D
Output: X_train, y_train, X_val, y_val

1:  Compute data overview statistics
2:  Remove duplicate rows from dataset
3:  Split dataset into:
        X_train, X_val, y_train, y_val
4:  Generate exploratory graphs (if enabled)
5:  Apply feature preprocessing on X_train and X_val
6:  Apply target preprocessing on y_train and y_val
```

```
7:  If is_report = True then
8:     Execute report generation
9: End if
10: Return X_train, y_train, X_val, y_val
```

4.5.7 Text Dataset Testing

Testing Preprocessing Steps

1. Load saved training metadata from `data_info.pkl` and recover the original `text_col` and `target_col`.
2. Validate that the test DataFrame contains the text column. If the target column is absent, enable feature-only mode.
3. Normalize string values by lowercasing the entire DataFrame using `lower_data`.
4. Fill missing text values with the empty string and transform test text using the saved TF-IDF transformer.
5. If target labels exist, fill missing labels with the saved training mode and apply the saved label encoder.
6. Return preprocessed text features and labels (or `None` for labels when feature-only mode is enabled).

Constructor Initialization and Artifact Loading: The testing constructor loads `data_info.pkl` from the saved folder path and recovers the original `text_col`, `target_col`, and `target_mode`. It verifies that the text column exists in the test DataFrame and sets `features_only=true` if the target column is missing.

Table 4.12: Text Testing Preprocessing Parameters

Parameter	Description
`df`	Input pandas DataFrame containing test examples.
`folder_path`	Directory path containing saved training artifacts and metadata.

Text Testing Preprocessing Parameters:

Feature Preprocessing: The feature preprocessing method fills missing values in the text column with empty strings and applies the loaded TF-IDF transformer from `training_preprocessor.pkl`. This yields the final sparse feature matrix for test time.

Target Preprocessing: If the test set contains the target column, missing targets are replaced with the saved training mode and then transformed with the loaded label encoder from `target_preprocessor.pkl`. If the target column is missing, the method returns `None` for the labels.

Common Preprocessing Flow: The test pipeline first lowercases all string values in the DataFrame, then executes feature preprocessing and target preprocessing sequentially. It returns the transformed text features and encoded labels.

Algorithm 4.9: Text Testing Preprocessing Pipeline

```
Input: test DataFrame D, saved folder path
Output: X_test, y_test or None

Load data_info.pkl and recover text_col, target_col, target_mode
Validate that text_col exists in D
If target_col not in D then set features_only = True
Lowercase all string values in D
Fill missing values in D[text_col] with empty string
Apply saved TF-IDF transformer to D[text_col]
If features_only then
    y_test = None
else
    Fill missing values in D[target_col] with target_mode
    Transform targets with the saved label encoder
End if
Return X_test, y_test
```

Saved artifacts used by text testing:

4.5.8 Image Dataset Training

The auto preprocessed image pipeline is designed to handle a vision task: image classification.

Table 4.13: Artifacts consumed by Text Testing Preprocessing

Artifact	Purpose
`data_info.pkl`	Contains saved metadata: `text_col`, `target_col`, and `target_mode`.
`training_preprocessor.pkl`	Saved TF-IDF vectorizer used to transform test text.
`target_preprocessor.pkl`	Saved LabelEncoder used to transform test labels.

Expected Dataset Structure The dataset is expected to follow the directory structure shown below. The validation folder is optional; if it exists.

```
Training/
  Cats/
  Dogs/

Validation/
  Cats/
  Dogs/
```

Classification image training preprocessing steps:

1. Initialize the constructor and validate input parameters.
2. Generate class-label mappings.
3. Load and validate images.
4. Create dataset overview.
5. Split training data into training and validation sets (optional).
6. Apply image preprocessing and augmentation.
7. Construct training and validation datasets.
8. Create DataLoaders for batch processing.
9. Generate preprocessing visualizations and reports.
10. Return processed training and validation DataLoaders.

Parameter	Description
training_folder_images	Path to training image folder (class-wise structure)
validation_folder_images	Path to validation image folder (optional)
folder_path	Output directory for saving reports, mappings, and graphs
split_training	Whether to split training folder into train/validation sets
val_size	Validation split ratio in range (0, 0.5]
random_state	Random seed for reproducibility
images_size	Image resize dimensions (height, width)
augment	Enable or disable data augmentation
batch_size	Batch size for DataLoader
augmentation_probability	Probability of applying augmentation in range [0,1]
horizontal_flip	Enable horizontal flip augmentation
vertical_flip	Enable vertical flip augmentation
rotation	Enable rotation augmentation
rotation_angle	Maximum rotation angle in degrees
brightness	Enable brightness augmentation
brightness_factors	Range for brightness adjustment
contrast	Enable contrast augmentation
contrast_factors	Range for contrast adjustment

Table 4.14: Classification Image Training Preprocessing Parameters

Parameter	Validation Rules
training_folder	Must not be None and must exist in filesystem
folder_path	Must be not None.
validation_folder	If provided, must exist and must be different from training_folder
split_training	If validation_folder is provided, must be False
val_size	Must be a float in range (0, 0.5]
random_state	Must be integer or None
images_size	Must be a tuple of two integers in range [32, 1024]
batch_size	Must be integer greater than 0
augment	Must be boolean
horizontal_flip	Must be boolean
vertical_flip	Must be boolean
rotation	Must be boolean
brightness	Must be boolean
contrast	Must be boolean
augmentation_probability	Must be float in range [0, 1]
rotation_angle	Must be numeric (int/float) and greater than or equal 0
brightness_factors	Must be a tuple of two numeric values (int/float)
contrast_factors	Must be a tuple of two numeric values (int/float)

Table 4.15: Validation Rules for Classification Image Training Preprocessing Parameters

Initialize the constructor and validate input parameters:

Generate class-label mappings: The system scans the training folder to identify all class subdirectories. Each class is assigned a unique numerical identifier, creating both class-to-label and label-to-class mappings. These mappings are stored for later use during training, evaluation, and prediction.

Load and validate images: Images are collected from each class folder and associated with their corresponding labels. Every image is validated to ensure that it can be successfully loaded and processed. Invalid or corrupted images are excluded from the dataset and recorded for reporting purposes.

Create dataset overview: A dataset overview is generated containing information about the available classes, image counts, invalid image counts, class mappings, and randomly selected image samples. This information is collected and stored for inclusion in the preprocessing report.

Split training data into training and validation sets (optional): If a `split_training`, the training data is divided into training and validation subsets according to the specified `val_size`. The split is performed using a fixed random seed to ensure reproducibility.

Apply image preprocessing and augmentation: Image preprocessing transformations are configured, including image resizing and optional augmentation techniques such as horizontal flipping, vertical flipping, rotation, brightness adjustment, and contrast adjustment. These transformations are applied when images are loaded from the dataset.

Construct training and validation datasets: Training and validation dataset objects are created using the processed image paths, labels, image size configuration, and augmentation settings. These datasets define how samples are loaded and transformed during training.

Create DataLoaders for batch processing: PyTorch DataLoaders are created from the dataset objects. The training DataLoader enables shuffling to improve learning, while the validation DataLoader preserves data order.

Generate preprocessing visualizations and reports: Several visualizations are generated to analyze the dataset and preprocessing results. Label summary graphs display class distributions and valid versus invalid image statistics. Random sample visualizations show representative images from each class. Additional visualizations display images after passing through the DataLoader to verify preprocessing and augmentation operations. All generated information is incorporated into the preprocessing report.

Return processed training and validation DataLoaders: The preprocessing pipeline concludes by returning the generated training and validation DataLoaders, which are subsequently used by the model training and evaluation procedures.

4.5.9 Image Dataset Testing

Testing Preprocessing Steps

1. Initialize the constructor and validate input parameters.
2. Validate input image files.
3. Load preprocessing metadata and class mappings.
4. Convert class names to numerical labels (optional).
5. Construct the testing dataset.
6. Create the testing DataLoader.
7. Return the testing DataLoader and invalid images.

Initialize the constructor and validate input parameters: The preprocessing pipeline initializes the testing configuration and validates the provided inputs. It verifies that the image path list is not empty, ensures that optional class names are

provided as a list, and confirms that the number of class names matches the number of image paths when labels are supplied.

Parameter	Description
image_paths	List of image file paths to be preprocessed and loaded for testing.
image_class_names	Optional list containing the class name corresponding to each image path. Used when labels are available for evaluation.
folder_path	Directory containing the preprocessing artifacts generated during training, including image size information and class mappings.
batch_size	Number of images processed together in each batch by the DataLoader.

Table 4.16: Parameters of ClassificationImageTestingPreprocessing

Parameter	Validation Rules
image_paths	Must be a non-empty list and contain at least one valid image.
image_class_names	If provided, it must be a list with the same length as image_paths.
folder_path	Must contain the files data_info.pkl and class2label_mapping.pkl generated during training.
batch_size	Used as the DataLoader batch size.
class names	If provided, every class name must exist in the training class-to-label mapping.

Table 4.17: Validation Rules for ClassificationImageTestingPreprocessing

Validate input image files: Each image path is checked to determine whether it is a valid image file. Valid image paths are collected for further processing, while invalid or unreadable images are stored separately. The preprocessing procedure terminates if no valid images are found.

Load preprocessing metadata and class mappings: The preprocessing metadata generated during training is loaded from disk. This includes the image size configuration and the class-to-label mapping used during model training. These artifacts ensure consistency between training and testing data preprocessing.

Convert class names to numerical labels (optional): If class names are provided, each class name is converted to its corresponding numerical label using the stored class-to-label mapping. An error is raised if a class name does not exist in the training classes.

Construct the testing dataset: A testing dataset is created using the validated image paths, optional labels, and image size configuration loaded from the training stage. Data augmentation is disabled to ensure that testing images remain unchanged.

Create the testing DataLoader: A PyTorch DataLoader is constructed from the testing dataset. The DataLoader processes images in batches without shuffling to preserve the original sample order during inference or evaluation.

Return the testing DataLoader and invalid images: The preprocessing pipeline returns the generated testing DataLoader together with a list of invalid image files that were excluded from processing.

4.6 AutoML Module

4.7 Deployment Module

4.8 Benchmark Platform Module

Chapter 5: System Testing and Verification

5.1 Testing Setup

The available experiments use identical train, validation, and test splits when comparing candidate pipelines. Timing, memory, output equivalence, and predictive accuracy are reported according to the module under test.

5.2 Testing Plan and Strategy

Module tests validate graph correctness, pipeline persistence, converter support, loop transformation, and output equivalence. Integration tests evaluate the end-to-end path from custom preprocessing code through native compilation and graph execution.

5.3 Accelera Pipe Module

The Accelera Pipe experiments compare three implementations under the same train, validation, and test split. The first implementation is a manually written scikit-learn loop over the candidate pipelines. The second uses `sklearn.pipeline.Pipeline` to represent the same candidates. The third uses Accelera Pipe with graph branches. Each candidate is selected using validation accuracy, and the final selected path is evaluated on the test set. This protocol checks whether the graph execution improves latency without changing the selected model or the measured predictive accuracy.

Table 5.1: Accelera Pipe latency scaling in seconds

Samples	sklearn	sklearn Pipeline	Accelera Pipe
1,000	0.06	0.05	0.34
5,000	2.47	2.12	2.02
10,000	2.48	2.48	1.79
50,000	23.96	23.25	14.81
100,000	77.79	77.10	67.04
250,000	803.56	803.01	494.64

Table 5.1 shows the main performance trend without compressing the font. At very small sizes, Accelera Pipe is slower because the native graph setup, branch creation, and Python/C++ boundary cost dominate the actual model computation. Once the dataset grows, those fixed costs become small compared with fitting multiple candidate

branches. At 50,000 samples, Accelera Pipe reduces runtime from 23.96 seconds to 14.81 seconds. At 250,000 samples, it reduces runtime from about 803 seconds to about 495 seconds, saving more than five minutes while selecting the same candidate.

Table 5.2: Accelera Pipe RSS memory scaling in MB

Samples	sklearn	sklearn Pipeline	Accelera Pipe
1,000	1.62	0.16	19.32
5,000	3.62	1.09	21.62
10,000	11.47	2.19	7.25
50,000	126.95	9.75	209.25
100,000	139.70	47.65	300.93
250,000	303.95	28.52	594.86

Table 5.2 shows the current cost of graph-level parallelism. Accelera Pipe uses more RSS memory because multiple branch states and intermediate arrays can exist at the same time. The memory optimizations in the methodology section, especially consumer-count based release, duplicate first-node sharing, and guarded preprocessing copies, target exactly this issue.

Table 5.3: Selected candidate and accuracy across sample sizes

Samples	Selected candidate	Validation accuracy	Test accuracy
1,000	MinMaxScaler $\rightarrow$ SVC	0.7900	0.8250
5,000	MinMaxScaler $\rightarrow$ SVC	0.9160	0.9100
10,000	MinMaxScaler $\rightarrow$ SVC	0.9385	0.9260
50,000	MinMaxScaler $\rightarrow$ SVC	0.9565	0.9598
100,000	StandardScaler $\rightarrow$ SVC	0.9675	0.9709
250,000	StandardScaler $\rightarrow$ SVC	0.9774	0.9768

Table 5.3 confirms that the speedup does not come from changing the search space. Accelera Pipe evaluates the same candidate pipelines and reports the same validation and test behavior. The selected preprocessing strategy changes from MinMax scaling to standard scaling only when the larger dataset makes StandardScaler with SVC the best validation candidate. This is expected because the candidate set and selection rule are fixed while the data distribution becomes better represented at larger sizes.

Table 5.4: Largest Accelera Pipe comparison

Implementation	Time (s)	RSS MB	VMS MB	Test accuracy
sklearn manual loop	803.56	303.95	315.41	0.9768
sklearn Pipeline	803.01	28.52	28.61	0.9768
Accelera Pipe	494.64	594.86	787.47	0.9768

Table 5.4 isolates the largest run: 150,000 training samples, 50,000 validation samples, and 50,000 test samples. Accelera Pipe keeps exactly the same selected path, StandardScaler followed by SVC, and the same test accuracy. The runtime drop is 308.91 seconds relative to the manual scikit-learn loop and 308.36 seconds relative to scikit-learn Pipeline. This result is important because it demonstrates the value of graph scheduling on realistic branch-heavy workloads. The memory columns also show the next engineering target: scikit-learn Pipeline is extremely memory efficient, while Accelera Pipe trades memory for parallel branch execution and native graph state.

5.4 Code Parallelizer Module

The OpenMP classifier was evaluated as a three-class loop classifier. The labels are `none`, `parallel_for`, and `reduction`. The classification report shows that the model is balanced enough to guide the rule layer, while the final pragma decision remains protected by dependency checks.

Table 5.5: OpenMP classifier report

Class	Precision	Recall	F1-score	Support
none	0.82	0.85	0.84	3048
parallel_for	0.86	0.81	0.83	2696
reduction	0.79	0.82	0.80	775
accuracy			0.83	6519
macro avg	0.82	0.83	0.83	6519
weighted avg	0.83	0.83	0.83	6519

Table 5.5 should be read as a guidance-quality result, not as a proof that every predicted loop is safe. The classifier is responsible for identifying likely parallelization opportunities. The rule layer is responsible for rejecting unsafe or unsupported cases. This two-stage design is useful because source code parallelization has correctness constraints that pure statistical classification should not decide alone.

Table 5.6: Hard parallelizer evaluation

File	Baseline ms	Parallel ms	Speedup	Output match
hard_eval_000.cpp	1786.28	195.47	9.138×	true
hard_eval_001.cpp	63.98	31.47	2.033×	true
hard_eval_002.py	42.66	7.67	5.563×	true

Table 5.6 evaluates harder programs from `data/hard_test_files`. The benchmark compiles and runs both the baseline and the generated parallelized code, then compares their outputs. The first file obtains the largest improvement because its loop body has enough work to amortize OpenMP thread overhead. The second file still improves, but the smaller absolute runtime limits the maximum speedup. The Python-origin file demonstrates the full converter path: Python subset to C++, then loop extraction, then pragma insertion. Across the three files, the evaluation extracted 11 loops, generated 8 pragmas, matched all 8 expected pragmas, reached average similarity 1.0, and obtained an average speedup of 5.578×.

Table 5.7: Linux parallelizer and Accelera Pipe integration benchmark

Mode	Samples	Time (s)
Python custom preprocessing function	500,000	6.83
End-to-end optimized run, first measured process	500,000	5.88
End-to-end optimized run, warmed method and module cache	500,000	0.08

Table 5.7 reports the Linux direct `examples/parallel_accpipe.py` run after normalizing the example execution directory to the repository root. The Python implementation takes 6.83 seconds on 500,000 samples. The first optimized process in the measurement sequence takes 5.88 seconds because it still pays process-local setup and cache lookup costs. The immediate repeated run takes 0.08 seconds after the Python-method cache and compiled-module cache are both warm. The validation in the example confirmed that the optimized output matched the Python output. This result shows why the integration needs two cache layers: one layer avoids repeating Python-to-C++ conversion and OpenMP insertion, while the lower compiled-module cache avoids recompiling the pybind extension.

Table 5.8: Linux direct example-script verification runs

Example	Baseline time (s)	Optimized time (s)	Validation
`accpipe_demo`	2.48	1.79	same accuracy, 0.9260
`parallel_accpipe`, run 1	6.83	5.88	outputs match
`parallel_accpipe`, run 2	6.83	0.08	outputs match
`code_parallelizer`, run 1	5.81	0.014	outputs match
`code_parallelizer`, run 2	6.05	0.015	outputs match
`save_load`, run 1	1.89	2.69 / 2.65	same accuracy, 0.268
`save_load`, run 2	2.53	2.20 / 2.14	same accuracy, 0.268

Table 5.8 summarizes the Linux examples invoked by `examples/Makefile`, but each file was run directly rather than through Make. Cache-sensitive files were repeated. The standalone `code_parallelizer_demo.py` result measures Python script execution against the generated OpenMP executable for `hard_eval_002.py`; the C path also reported about 0.26–0.27 seconds of compilation time, outside the listed executable runtime. The save/load example verifies persistence rather than pure acceleration: both sklearn and Accelera load persisted state and produce the same accuracy, while the second Accelera run is slightly faster than the sklearn baseline.

Table 5.9: Windows direct example correctness and path timing

Example	Run/cache state	Baseline (s)	Accelera/C path (s)	Validation
`accpipe_demo`	normal	1.75 / 1.78	1.03	same test accuracy, 0.9260
`parallel_accpipe`	isolated cache, run 1	6.66	6.00	outputs match
`parallel_accpipe`	isolated cache, run 2	6.58	0.09	outputs match
`code_parallelizer`	isolated cache, run 1	5.59	1.04	outputs match
`code_parallelizer`	isolated cache, run 2	5.61	1.05	outputs match
`save_load`	normal	0.57	0.51 / 0.53	same accuracy, 0.268

Table 5.10: Windows compilation and process overhead

Example	Run/cache state	Compile/link (s)	Wall time (s)	Interpretation
`accpipe_demo`	normal	–	6.53	graph example startup
`parallel_accpipe`	isolated cache, run 1	included	14.37	cold compile path
`parallel_accpipe`	isolated cache, run 2	cached	8.33	warm cache path
`code_parallelizer`	isolated cache, run 1	0.87	8.74	temporary executable
`code_parallelizer`	isolated cache, run 2	1.04	9.00	temporary executable
`save_load`	normal	–	2.71	persistence example

Tables 5.9 and 5.10 report the same Makefile example set on Windows. The first table keeps the user-visible comparison: baseline runtime, optimized path runtime, and validation result. The second table separates compilation and process-level overhead so the timing table remains readable. The examples were run one by one

in Makefile order. The cache-sensitive examples were then repeated under an isolated Accelera cache so that the first run represents cold cache behavior and the second run represents warm cache behavior. The strongest cache effect appears in `parallel_accpipe.py`: the cold end-to-end Accelera path takes 6.00 seconds because it must convert the function, select loops, compile a pybind extension, link it, and load it into Python. The immediate warm-cache run takes 0.09 seconds because the processed-code cache, method cache, and compiled-module cache avoid repeating those setup stages.

The Windows `code_parallelizer_demo.py` runs show a different limitation. The generated C/OpenMP executable is compiled into a temporary directory on every run, so the second run does not reuse the final linked program. The measured C executable subprocess takes about 1.04 seconds, while the instrumented main-body timer inside the generated program reports only about 0.024–0.025 seconds. This gap is Windows process startup, OpenMP runtime initialization, dynamic library loading, and executable loading overhead. The compile/link step itself adds about 0.87–1.04 seconds. On Linux, the same benchmark has lower launch and linking overhead in the reported measurements, so executable runtime is closer to the timed main-body work. For this reason, Windows results should be interpreted as end-to-end toolchain latency results, not only as loop-kernel speed results.

5.5 AutoPreprocessing Module

5.5.1 Experimental Setup

Experimental Environment All experiments were implemented in Python using Pandas, NumPy, and Scikit-learn for tabular data processing and model evaluation. AutoCleanML was used as the baseline preprocessing framework, while Accelera was used as the proposed automated preprocessing system. The experiments were executed under identical conditions.

Datasets A total of 16 tabular datasets were used, collected from Kaggle. The datasets include both classification and regression tasks with varying sizes and feature complexities.

Table 5.11: Summary of Datasets Used in Experiments

Dataset	Shape	Task Type
startup_founder_burnout_2026	(50000, 29)	Classification
healthy_diet_calorie_intake	(6000, 15)	Classification
ai_student	(50000, 16)	Classification
gaming_addiction	(250, 49)	Classification
Teen_Mental_Health_Dataset	(1200, 13)	Classification
student_exam_performance_dataset	(10000, 17)	Classification
heart	(302, 14)	Classification
Dataset_spine	(310, 7)	Classification
diabetes_dataset	(100000, 31)	Classification
student_placement_synthetic	(100000, 18)	Classification
Titanic-Dataset	(891, 12)	Classification
customer_purchase_data	(1388, 9)	Classification
global_ai_jobs	(90000, 35)	Regression
CarPrice_Assignment	(205, 26)	Regression
Concrete_Data_Yeh	(1005, 9)	Regression
Housing	(545, 13)	Regression

Compared Frameworks Two preprocessing tools were evaluated:

AutoCleanML: A fully automated data cleaning AutoCleanML Python library, that performs missing value handling, encoding, and feature preparation.

Accelera: A proposed automated preprocessing tools supporting tabular, image, and text data, with additional reporting and visualization capabilities.

Preprocessing Configuration Identical experimental settings were used across both tools. A validation size of 20% and a random state of 42 were applied for all datasets. For classification tasks, stratified sampling was used to preserve class distribution.

Apart from specifying the target column and excluding unnecessary columns when required, all preprocessing parameters were kept at their default values for both frameworks.

Results and Discussion

Classification Results The performance of both frameworks was evaluated on multiple classification datasets using F1-macro score as the evaluation metric.

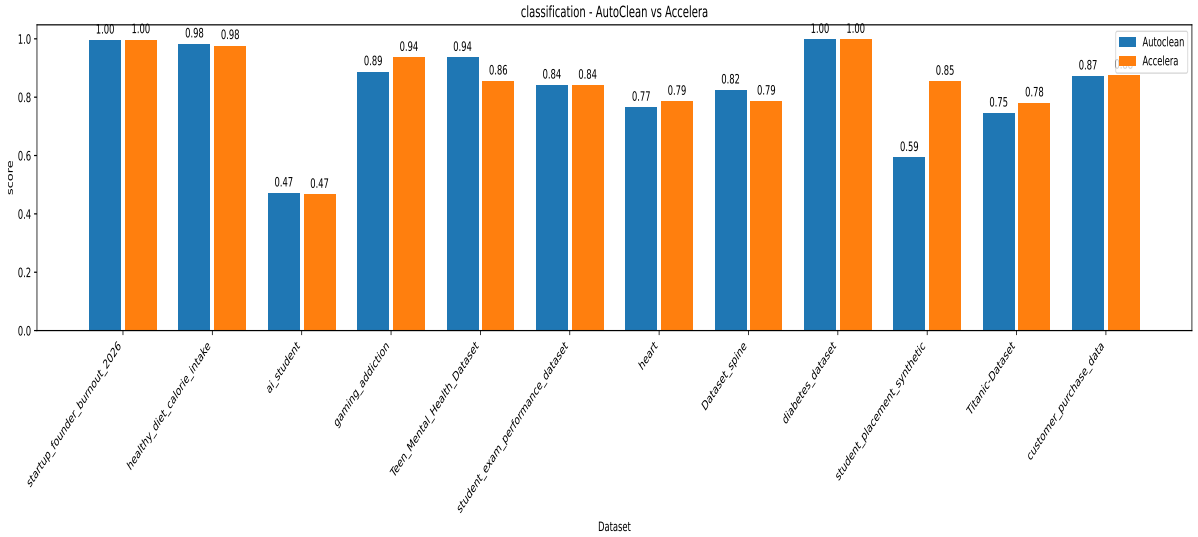

Figure 5.1: Classification Performance Comparison (F1-Macro Score)

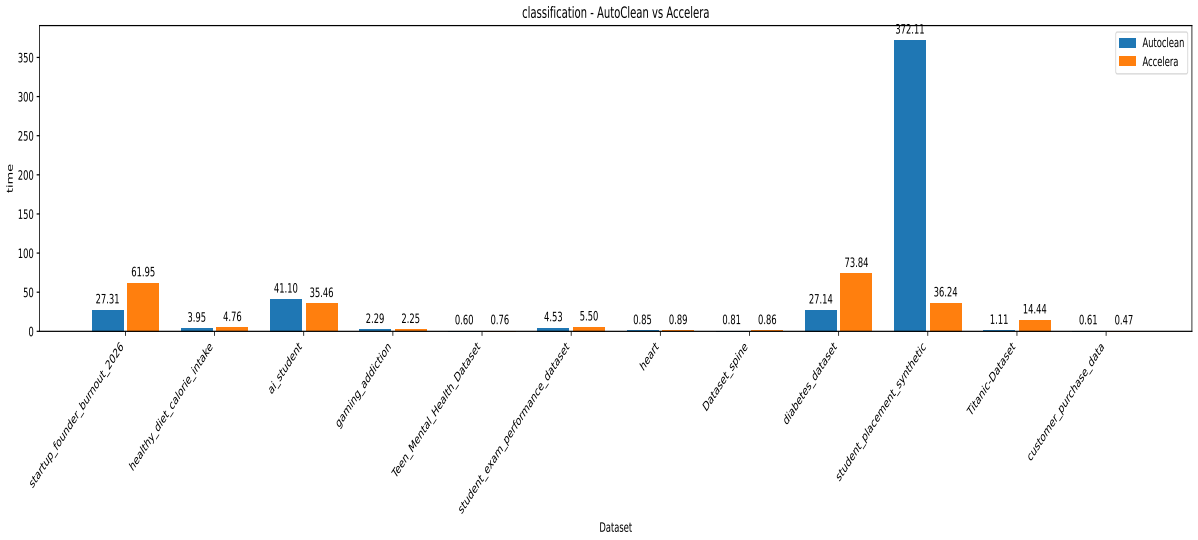

Figure 5.2: Classification Preprocessing Time Comparison

Regression Results Regression performance was evaluated using the R^2 score.

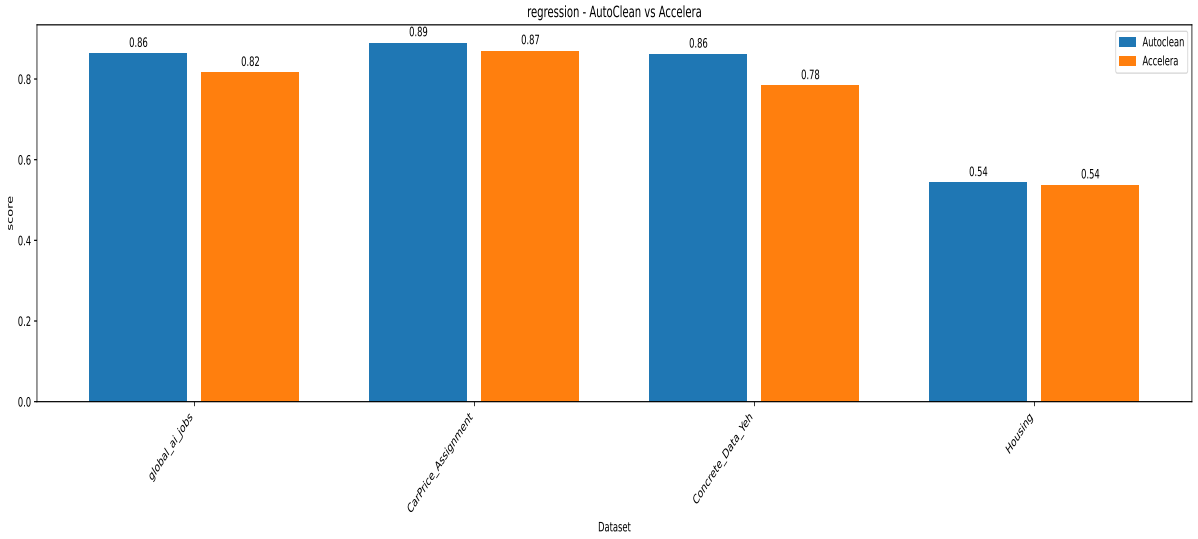

Figure 5.3: Regression Performance Comparison (R^2 Score)

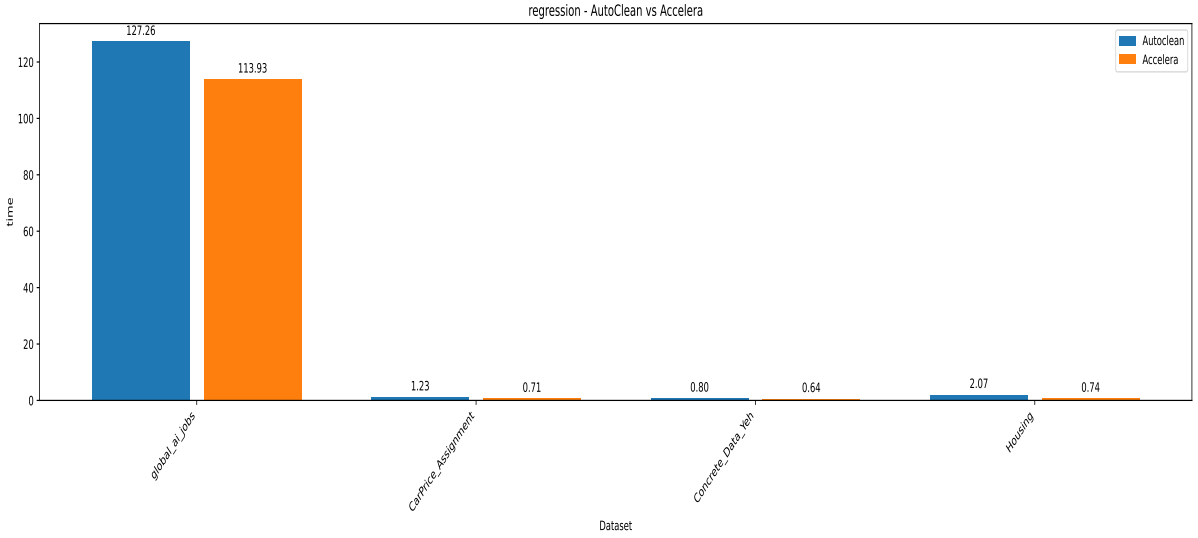

Figure 5.4: Regression Preprocessing Time Comparison

5.6 AutoML Module

5.7 Deployment Module

5.8 Benchmark Platform Module

Chapter 6: Conclusions and Future Work

6.1 Faced Challenges

The main implementation challenges were preserving correctness while sharing branch computations, releasing intermediate arrays only after their final consumer, and limiting generated OpenMP code to loops that pass dependency and conversion checks. The experiments also identify memory usage during branch-level parallel execution as the main remaining graph-backend challenge.

6.2 Gained Experience

6.3 Conclusions

Accelera Pipe provides a graph-based execution model for branch-heavy machine-learning pipelines and supports fitted executed graphs. The Code Parallelizer provides a source-to-source path from supported Python or C++ loops to OpenMP-annotated C++ using converter rules, loop feature extraction, classifier guidance, and safety checks. The available experiments show runtime improvements on large branch-selection workloads while preserving selected models, accuracy, and output equivalence.

6.4 Future Work

Future work should continue reducing graph memory consumption while preserving parallel branch execution, and should extend safe converter and loop-analysis coverage without weakening correctness checks.

References

Chapter A: Development Platforms and Tools

The implemented modules use Python, C++, PyBind11, CMake, Clang-based AST analysis, OpenMP, NumPy, and Scikit-learn. The available experiments include Linux and Windows execution paths.

Chapter B: Use Cases

Chapter C: User Guide

Chapter D: Code Documentation

Chapter E: Feasibility Study